

SC102 L01: C# 进阶（上）——详细学习笔记

涵盖：内存与地址 | 堆与栈 | 值类型与引用类型 | 字符串机制 | ref 关键字 | 装箱与拆箱 | 垃圾回收机制 (GC) | C#/Mono/.NET | C# 工作原理

1 课程概览

SC102 (Unity 游戏开发 I) 是 Silverjay 工作室游戏开发系列课程的第二部。本课程分为四个单元：技术基础、物理与动画、UI 组件编程、玩法系统编程。第一单元”技术基础”涵盖 C# 进阶、设计模式与数据结构，为后续的 Unity 开发打下扎实的编程基础。本课 (L01) 聚焦 C# 进阶的上半部分，核心主题是内存管理和类型系统。 [p.2-7]

2 内存与地址 (Memory & Address)

2.1 数据存放在哪里？

当我们编写代码时，定义的每一个变量——无论是一个简单的整数还是一个复杂的类实例——都被存储在电脑的内存 (Memory) 中。内存本质上是一块由 0 和 1 组成的巨大数字序列，其中的数据不是永久保存的（这也解释了为什么每次重新开始游戏时变量会恢复到初始值）。 [p.9]

2.2 地址 (Address) 的概念

当定义一个变量时，该变量的值会被编码为一串二进制数字并存入内存的某个位置，这个位置就称为变量的地址 (Address)。例如：

```
int a = 47;
```

在内存中，变量 a 的值可能表示为：00000000 00000000 00000000 00101111（精确的二进制表示因机器而异）。这串二进制中第一个”0”所在的内存位置就是 a 的地址。 [p.9]

2.3 为什么需要区分堆和栈？

对于内存管理而言，并非所有数据都是”平等的”。我们希望把较小的、访问频繁的数据放在一个访问速度快但空间有限的内存区域，而把较大的、生命周期不确定的数据放在一个访问较慢但空间很大的区域。这个需求催生了堆 (Heap) 和栈 (Stack) 两种内存区域的设计。 [p.9]

3 堆与栈 (Heap & Stack)

3.1 栈 (Stack)

栈是一块较小的内存区域，大小固定（通常在 1-2 MB 之间），主要存储**本地变量 (Local Variables)**和**值类型数据**。栈的访问速度极快，原因在于其 LIFO（后进先出）的数据结构和 CPU 缓存友好性。当函数被调用时，其局部变量被压入栈顶（push）；函数返回时，这些变量自动弹出（pop）——无需任何垃圾回收介入。[p.10]

栈上存储的数据类型包括：

- 值类型变量：int, float, double, bool, long, char 等基本数据类型
- 结构体 (struct)：如 Vector3, Quaternion, Color
- 枚举类型 (enum)
- 引用类型变量的引用本身（注意：引用本身存在栈上，但它指向的数据在堆上）

3.2 堆 (Heap)

堆是一块较大的内存区域，大小随着程序运行不断增长，最大可达绝大部分进程可用内存。堆主要用于存储用 new 关键字创建的**类实例 (Class Instances)**和其他引用类型的数据。由于堆上的内存分配需要在较大的空间中搜索可用块，且涉及 GC 管理，因此**堆的访问速度显著慢于栈**。[p.10]

堆上存储的数据类型包括：

- 类的实例 (class)：如 GameObject, Transform, MonoBehaviour
- 字符串 (string)
- 数组 (array)：无论元素是值类型还是引用类型
- 委托 (delegate)
- 接口类型 (interface)

3.3 堆与栈的核心区别总结

特性	栈 (Stack)	堆 (Heap)
大小	固定 (1-2 MB)	动态增长
访问速度	极快 (LIFO)	较慢 (需搜索 +GC)
存储内容	值类型、本地变量、引用	类实例、数组、字符串
管理方式	自动 (函数返回时清栈帧)	GC (垃圾回收器)
典型用途	小数据、临时变量	大对象、生命周期不可预测

理解堆栈对于我们编程的意义：栈上的变量被自动管理，而堆上的变量需要由 GC 管理。因此我们在写代码时应当尽量减少堆分配 (heap allocation)，比如：避免在 Update() 中频繁 new 对象、优先使用结构体做临时数据、使用对象池复用 GameObject 等。[p.10]

4 值类型与引用类型 (Value Type & Reference Type)

4.1 值类型 (Value Type)

值类型直接存储数据本身。变量之间的传递是通过拷贝 (copy) 的方式进行的。这意味着，将一个值类型变量赋值给另一个变量时，会把数据完整地复制一份，两个变量此后互不影响。[p.11]

示例：

```
int a = 7;
int b = a;    // b获取了a中"7"的一份拷贝
b = -1;       // 修改b不会影响a
// a仍然是7, b变成了-1
```

在这个例子中，a 中直接存储了数字 7，而 b = a 是把 a 中的 7 拷贝了一份赋值给 b。a 和 b 是两个完全独立的变量，存放在栈上的不同位置。[p.11]

值类型数据存储栈 (Stack) 上。这样设计的原因：

- 值类型通常较小 (int 只有 4 字节)，适合放在有限的栈空间中
- 栈的 LIFO 特性天然支持函数调用中的局部变量生命周期
- 不需要 GC 参与管理，分配和释放效率极高

常见的值类型：

- 所有基本数据类型：int, float, double, bool, long, char, byte, short 等
- 枚举类型 (enum)
- 结构体 (struct)：如 Unity 中的 Vector2, Vector3, Quaternion, Color, Rect 等

4.2 引用类型 (Reference Type)

引用类型不直接存储数据本身，而是存储指向数据所在地址的引用 (指针)。变量之间的传递只拷贝引用 (地址)，不拷贝实际数据。因此自始至终只有一份实际数据。[p.12]

示例：

```
Transform tf1 = object1.transform;
Transform tf2 = tf1;
tf2.position = new Vector3(1, 0, 0);
// 此时 object1 的位置也变成了 (1, 0, 0)!
```

分析：tf1 和 tf2 中都只存储了指向 object1.transform 所在内存的地址。tf2 = tf1 这行代码仅仅把地址拷贝了一份给 tf2，两个变量现在指向的是同一个 Transform 实例。因此通过 tf2 修改 position，object1 的位置也随之改变。[p.12]

引用类型本身 (即引用/地址) 存储在栈上，但它所指向的数据存储在堆上。这样设计的原因：

- 引用类型数据通常较大（一个 `GameObject` 可能包含数十个组件），不适合放在有限栈空间
- 引用类型数据的生命周期通常较长且不可预测（可能在整个游戏过程中都存在），不符合栈的后进先出特性
- 将实际数据放在堆上，只将轻量的引用放在栈上，兼顾了访问效率和内存容量

所有引用类型都继承自 `System.Object` 类。 [p.12]

常见的引用类型：

- 类（`class`）: `MonoBehaviour`, `GameObject`, `Transform`, `Component` 等
- 字符串（`string`）
- 数组（`array`）: 无论元素类型
- 委托（`delegate`）
- 接口（`interface`）

4.3 练习题解答

[p.13]

题 1：

```
Vector3 v = new Vector3(1, 1, 1);
Vector3 u = v;
u.x += 1;
print(v); // 输出 (1, 1, 1)
```

解释： `Vector3` 是一个 `struct`，属于值类型。`u = v` 时发生了值拷贝，`u` 和 `v` 是完全独立的两个变量。修改 `u.x` 不影响 `v`。因此 `v` 仍然是 (1, 1, 1)。

题 2：

```
GameObject go = new GameObject("Object 1");
Rename(go, "cube");
print(go.name); // 输出 "cube"

void Rename(GameObject go, string name) {
    go.name = name;
}
```

解释： `GameObject` 是一个 `class`，属于引用类型。函数参数 `go` 接收的是外部 `go` 变量中存储的引用（地址）的拷贝，但这个拷贝仍然指向同一个 `GameObject` 实例。所以在函数内通过引用修改 `go.name`，修改的是同一个堆上的实例，外部 `go.name` 也随之变为“cube”。

题 3：

```
GameObject go1 = new GameObject("Object 1");
GameObject go2 = new GameObject("Object 2");
Copy(go1, go2);
print(go1.name); // 输出 "Object 1"
```

```
void Copy(GameObject go1, GameObject go2) {
    go1 = go2;
}
```

解释：这里最容易混淆！函数内的 `go1` 是外部 `go1` 中引用的拷贝（栈上的地址被复制了一份传入了函数）。在函数内执行 `go1 = go2`，只是让函数内的局部变量 `go1` 指向了 `go2` 指向的实例，但外部的 `go1` 变量中存储的地址完全没有改变。所以外部的 `go1` 仍然指向“Object 1”。这就是“引用传递”和“值传递引用”的区别——如果不使用 `ref` 关键字，引用本身也是按值传递的（传递的是地址的拷贝）。

5 C# 中的字符串机制（String in C#）

5.1 字符串的“奇怪”行为

[p.14]

考虑以下代码：

```
string s = "123";
Append(s);
print(s); // 输出 "123", 而不是 "123a"!
```

```
void Append(string s) {
    s += "a";
}
```

为什么 `s` 仍然是“123”？如果 `string` 是引用类型，按之前的分析，函数内对同一块堆内存的修改应该对全局可见才对？要回答这个问题，需要理解 `string` 的两个核心特性：**不变性（Immutability）**和**字符串驻留（String Interning）**。

5.2 不变性（Immutability）

[p.15]

C# 中的 `string` 是不可变的（immutable）。一旦一个 `string` 对象被创建，它永远不能被改变。任何对 `string` 的“修改”操作，实际上都是创建了一个全新的 `string` 对象。

例如：

```
string s = "123";
```

```
s += "4";
```

这段代码实际做了以下事情：

1. 在堆上创建字符串 "123", s 指向它
2. 计算 s + "4", 在堆上创建一个全新的字符串 "1234"
3. 将 s 重新指向新字符串 "1234"
4. 原来的字符串 "123" 不再被任何变量引用, 成为”垃圾”等待 GC 回收

现在可以分析 Append 的例子了：

```
void Append(string s) {           // s 是外部s中引用的拷贝
    s += "a";                     // 等价于 s = s + "a"
}                                 // 1. s+"a" 创建新string "123a"
                                // 2. 函数内的局部s指向"123a"
                                // 3. 外部的s仍然指向"123"!
                                // 4. 函数返回后, 局部s消失, "123a"变成垃圾
```

为什么字符串被设计成不可变的？

- **线程安全**：不可变对象天然就是线程安全的，多个线程可以同时读取同一个字符串而无需加锁
- **安全性**：字符串经常用于存储敏感信息（如文件路径、连接字符串），不可变性防止其被意外修改
- **字符串驻留 (String Interning)**：因为不可变，多个引用可以安全地指向同一个字符串实例
- **哈希一致性**：字符串常用作字典的键，不可变性保证了哈希值永远不会改变

5.3 字符串驻留 (String Interning)

[p.16]

由于字符串的不可变性，C# 内置了字符串驻留 (String Interning) 机制来优化内存。它类似于一个”字符串常量池”：在编译时（以及通过 `string.Intern()` 在运行时），内容相同的字符串字面量会被合并为一个实例。

```
string str1 = "Hello";
string str2 = "Hello";
bool areEqual = Object.ReferenceEquals(str1, str2);
// areEqual == true!
// str1和str2指向堆上完全相同的地址！
```

驻留机制的好处：

- 避免在内存中存储大量重复的字符串副本
- 字符串相等比较（==）可以先比较引用，如果引用相同则直接返回 true，极大提升性能
- 在 Update() 中频繁使用相同的字符串字面量时不会产生额外垃圾

注意：只有在编译时确定的字符串字面量（string literal）才会被自动驻留。运行时动态创建的字符串（如通过 StringBuilder 构造、+ 拼接得到的结果）不会被自动驻留，除非手动调用 string.Intern()。

6 ref 关键字

6.1 为什么需要 ref?

[p.17]

C# 中，函数的参数默认是按值传递（pass by value）的。对于值类型参数，函数内的修改不影响外部变量；对于引用类型参数，函数内修改引用本身（改变指向哪个对象）不影响外部变量，但通过引用修改对象内部状态是可见的。

ref 关键字允许我们按引用传递（pass by reference）参数。使用 ref 后，函数参数直接使用外部变量的引用，而非其拷贝。

6.2 值类型的 ref 示例

```
// 不使用ref
int a = 1;
AddOne(a);
print(a); // a = 1（不变）

void AddOne(int a) { a++; } // 修改的是a的拷贝

// 使用ref
int a = 1;
AddOne(ref a);
print(a); // a = 2（变了！）

void AddOne(ref int a) { a++; } // 直接操作外部的a
```

6.3 引用类型的 ref 示例

```
// 不使用ref
string s = "123";
Append(s);
```

```
print(s); // s = "123" (不变, 因为string的不变性)
```

```
void Append(string s) { s += "a"; }
```

```
// 使用ref
```

```
string s = "123";
```

```
Append(ref s);
```

```
print(s); // s = "123a" (变了!)
```

```
void Append(ref string s) { s += "a"; }
```

对于引用类型，`ref` 使得函数不仅可以修改对象的内部状态，还可以修改引用本身（即让外部变量指向完全不同的对象）。在非 `ref` 模式下，函数内 `s += "a"` 只修改了局部引用指向哪里；而在 `ref` 模式下，修改的是外部变量 `s` 本身。

6.4 练习：写一个交换两个 *int* 变量值的函数

```
void Swap(ref int a, ref int b) {
```

```
    int temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
}
```

```
// 使用
```

```
int x = 5, y = 10;
```

```
Swap(ref x, ref y);
```

```
// x = 10, y = 5
```

关键点：如果不加 `ref`，`a` 和 `b` 在函数内只是外部变量的拷贝，交换它们不会影响外部的 `x` 和 `y`。

7 装箱与拆箱 (Boxing & Unboxing)

7.1 概念解释

[p.19]

装箱 (**Boxing**) 和拆箱 (**Unboxing**) 是值类型和引用类型之间转换的操作。由于所有引用类型都继承自 `System.Object`，所以任何值类型都可以被当作 `object` 处理——但这种转换不是免费的。

- **装箱 (Boxing)：**从值类型转换为引用类型 (`object` 或接口类型)。在堆上分配内存，将值类型的数据复制到堆上，然后返回指向堆上对象的引用。

- **拆箱 (Unboxing)**: 从引用类型转换回值类型。检查堆上对象是否确实是目标值类型，然后将其数据从堆复制回栈。

示例:

```
int a = 7;
object o = a;    // 装箱 (Boxing): 在堆上分配空间, 将7复制进去, o指向它

object o = 42;
int a = (int)o;  // 拆箱 (Unboxing): 先检查o中确实是int, 然后将42从堆复制到栈上a
```

7.2 为什么装箱和拆箱是”昂贵”的操作？

- **装箱**需要在堆上分配新内存 (涉及 GC), 并将值类型数据从栈复制到堆——相比简单的栈操作昂贵得多
- **拆箱**需要进行类型检查 (确保安全), 然后将数据从堆复制回栈
- 装箱会产生”垃圾” (堆上的临时对象), 增加 GC 压力

在 Unity 开发中应避免装箱的场景:

- 将值类型赋值给 `object` 类型变量
- 调用接受 `object` 参数的方法时传入值类型
- 将值类型放入非泛型集合 (如 `ArrayList`; 应使用泛型 `List<int>` 替代)
- 字符串格式化时 (`string.Format()` 的参数是 `object` 类型, 传入值类型会装箱)

8 垃圾回收机制 (Garbage Collection, GC)

8.1 为什么需要 GC?

[p.20]

我们一直在用 `new` 在堆上创建新对象, 但从未手动删除它们。如果没有 GC, 堆内存会不断增长直到耗尽。C# 的垃圾回收器 (**Garbage Collector, GC**) 自动检测并清理不再被引用的堆上对象, 释放内存。

GC 管理的内存 (堆) 被分为三个代 (**Generation**):

- **Generation 0 (第 0 代)**: 主要存放刚被创建的临时对象和较小的类实例。GC 清理 Gen0 的频率最高 (因为大部分临时对象的生命周期很短)。
- **Generation 1 (第 1 代)**: 一个”缓冲区”。从 Gen0 中存活下来的对象被提升到 Gen1 (说明它们生命周期较长, 仍在使用中)。清理频率低于 Gen0。

- **Generation 2 (第 2 代)**: 存放长期需要的大对象（如静态变量、单例实例等）。清理频率最低。

重要规则: 当 GC 清理第 k 代时, 所有小于 k 的 generation 也会被一并清理。因此清理 Gen2 时相当于进行一次完整的 (full) 垃圾回收, 开销最大。[p.20]

8.2 垃圾回收的三个阶段

阶段 1: 标记阶段 (Marking Phase) [p.21]

GC 从一组“根引用”(root references) 出发——包括静态变量、当前活跃的栈上的本地变量、CPU 寄存器中的引用等——通过递归地追踪引用关系, 构造出一个引用树 (Reference Tree)。所有从根节点可达 (reachable) 的对象被标记为“仍在使用的 (in use)”, 这些对象在本次清理中存活。

伪代码说明标记算法的递归本质:

```
void Mark(ObjectRef o) {
    if (!InUseList.Exists(o)) {
        InUseList.Add(o);           // 标记为"使用中"
        List refs = GetAllChildReferences(o); // 获取o引用的所有其他对象
        foreach (ObjectRef childRef in refs) {
            Mark(childRef);         // 递归标记
        }
    }
}
```

阶段 2: 回收阶段 (Collection Phase) [p.22]

GC 遍历堆上的所有对象, “删除”所有未被标记的物体 (即标记阶段未发现的、不可达的对象)。所谓“删除”并不是真正抹去数据, 而只是将该对象所在的内存区域标记为“未使用”, 后续分配时可以直接覆盖。因此原来的二进制数据可能还残留在内存中。

阶段 3: 压缩阶段 (Compacting Phase) [p.22]

回收阶段后, 堆中会出现大量碎片/空隙 (被“删除”的物体留下了空洞)。GC 将存活的对象重新排列, 使它们“沉到堆底”、紧密排列, 消除内存碎片。在此过程中, 对象之间的相对排列顺序不变。压缩完成后, 新对象的分配可以在连续的空闲内存上进行, 效率更高。

压缩对性能的影响: 压缩需要移动对象并更新所有引用, 对于大对象 (尤其是 Gen2 中的大型数组) 成本很高。因此大型对象通常分配到独立的 Large Object Heap (LOH), 不参与压缩。

8.3 GC 何时被触发?

[p.22]

1. 当可用内存快满时, 或内存占用超过 GC 设定的阈值
2. 手动调用 GC.Collect() 方法 (一般不推荐, 因为 GC 最清楚何时回收最合适)

3. Gen0 的分配预算用完时（Gen0 有一个固定大小的预算，新分配超出预算即触发 Gen0 GC）

8.4 GC 性能优化：减少”垃圾”的产生

[p.23]

GC 虽然方便，但清理过程很耗性能。作为开发者，我们应当谨慎地”生产垃圾”。

示例 1：字符串拼接

```
// 坏写法：每次循环创建一个新string对象，产生大量垃圾
string s;
for (int i = 0; i < 1000; i++) {
    s += "a";
}
// 在string不变性的作用下，每次+=都会在堆上创建全新的string，
// 旧的string变成垃圾。1000次循环 1000个垃圾对象！
```

示例 2：Unity 中的顶点访问

```
// 坏写法
for (int i = 0; i < mesh.vertices.Length; i++) {
    float x = mesh.vertices[i].x;
    float y = mesh.vertices[i].y;
    float z = mesh.vertices[i].z;
}
// 问题：每次访问mesh.vertices时，Unity会返回vertices数组的一份拷贝！
// 这段代码在循环中4次访问mesh.vertices，产生了4份数组拷贝作为垃圾。
```

```
// 好写法
Vector3[] vertices = mesh.vertices; // 只拷贝一次
for (int i = 0; i < vertices.Length; i++) {
    float x = vertices[i].x;
    float y = vertices[i].y;
    float z = vertices[i].z;
}
```

Unity 中减少 GC 压力的通用策略：

- 避免在 Update()/FixedUpdate() 中 new 对象或拼接字符串
- 使用缓存变量（cache）：将频繁访问的属性（如 transform）缓存到本地变量
- 使用对象池（Object Pool）复用 GameObject，而非 Instantiate/Destroy
- 使用 StringBuilder 替代 + 拼接字符串
- 使用非分配版本的 API（如 Physics.RaycastNonAlloc）

9 C#、Mono、.NET

9.1 .NET Framework

[p.25]

.NET 是微软为 Windows 推出的程序开发框架，包含大量常用类库和一个称为公共语言运行时（Common Language Runtime, CLR）的运行环境。CLR 提供以下核心功能：

- 将 C#/VB.NET/F# 代码翻译为机器指令（通过 JIT 编译）
- 内存管理（包括 GC）
- 多线程管理
- 异常处理
- 类型安全检查

9.2 Mono —— .NET 的多平台实现

.NET 虽然强大，但最初主要是为 Windows 设计的。为了让 .NET 应用能在其他操作系统（macOS、Linux、iOS、Android 等）上运行，**Mono** 诞生了。Mono 是 .NET Framework 的一个开源、跨平台的实现，它包含：

- C# 编译器（将 C# 代码编译为 IL）
- Mono Runtime（与 CLR 对应，提供运行时环境）
- .NET 兼容类库 + Mono 独有的类库

补充知识（2024 年更新）：微软在 2016 年收购 Xamarin 后推出了 .NET Core（后更名为 .NET 5/6/7/8/9），统一了 .NET Framework 和 Mono。现代 Unity 版本（2021 LTS+）正在逐步从 Mono 迁移到现代的 .NET（CoreCLR）。但理解 Mono 仍然重要，因为：

- MonoBehaviour 这个关键类名来源于 Mono Runtime
- 项目中很多旧代码和文档仍然引用 Mono
- Unity 的 IL2CPP 技术也是从 Mono 的 IL 出发

9.3 MonoBehaviour 名称的由来

[p.25]

MonoBehaviour = "Mono" Runtime + "Behaviour"（物体行为）。所以 MonoBehaviour 的含义是：基于 Mono Runtime 的物体行为脚本组件。Unity 中所有挂载到 GameObject 上的脚本都继承自 MonoBehaviour，正是因为它需要依赖 Mono Runtime 来管理脚本的生命周期（Awake、Start、Update 等回调）。

10 C# 程序：从代码到指令

10.1 编译三部曲

[p.26]

一段 C# 代码从编写到在电脑上运行，需要经过三个步骤、两层编译：

步骤 1：C# 源代码（Source Code）——程序员手写、人类可读的高级语言代码。

步骤 2：编译为 IL/CIL（Common Intermediate Language，中间语言）——当我们编译（Build）C# 项目时，C# 编译器（csc/Roslyn）将源代码转换为平台无关的 IL 代码（在 Windows 上生成 .exe 或 .dll 文件，在其他平台上是对应的可执行格式）。IL 是一种介于高级语言和机器码之间的中间表示。

步骤 3：JIT 编译为机器码（Machine Code）——当我们运行程序时，CLR/Mono Runtime 中的 JIT 编译器（Just-In-Time Compiler）将 IL 代码逐段编译为当前 CPU 架构的原生机器指令。这一步发生在程序运行时，只编译实际被调用的方法（而非一次性编译全部代码）。

C#源码	编译 →	IL/CIL	JIT编译 →	机器码
(.cs)	(Roslyn)	(.exe/.dll)	(CLR/Mono)	(CPU直接执行)

10.2 为什么需要 IL 这一步？

- **跨平台：**同一份 IL 代码可以在任何有 CLR/Mono Runtime 的平台上运行
- **语言互操作：**所有面向 .NET 的语言（C#、VB.NET、F#）都编译为相同的 IL，因此可以无缝互相调用
- **JIT 优化：**JIT 编译器根据实际运行环境和 CPU 特性进行优化（如内联、循环展开），生成的机器码可能比 AOT 编译更高效

10.3 Unity 中的特殊情况：IL2CPP

在移动平台（iOS）和部分主机平台上，由于不允许 JIT 编译（安全/性能政策限制），Unity 使用 **IL2CPP（Intermediate Language To C++）** 技术：将 IL 代码转换为 C++ 代码，然后用平台原生编译器将 C++ 编译为机器码。这是一种 Ahead-Of-Time（AOT）编译方式。

11 课后习题详解

[p.27]

11.1 题 1：浅述堆和栈的异同。理解堆和栈对于我们编程有什么用处？

相同点：堆和栈都是内存中的区域，用于存储程序运行时产生的数据。

不同点：

- 栈大小固定（1-2MB），堆大小动态增长
- 栈访问速度极快（LIFO 结构），堆访问较慢
- 栈存储值类型和本地变量，堆存储引用类型实例
- 栈自动管理（函数返回即清理），堆由 GC 管理
- 栈上数据生命周期短（函数作用域内），堆上数据生命周期不确定

编程启示：

- 了解栈的有限大小——不要在栈上分配超大数组或深度递归
- 了解堆分配的成本——减少不必要的 `new`，尤其在热点路径（Update 等）
- 了解 GC 的工作方式——控制“垃圾”的产生量，使用对象池和缓存
- 选择正确的类型——小数据、临时数据优先使用 `struct`（值类型），复杂对象、长生命周期数据使用 `class`（引用类型）

11.2 题 2: 分析红色和蓝色代码

红色代码（错误写法）：

```
class A { int num; }

A instance1 = new A();
A instance2 = instance1;
instance2.num = 5;
```

这段代码功能正确但有一个概念陷阱：`instance1` 和 `instance2` 指向同一个 A 实例（引用类型）。修改 `instance2.num` 也会影响 `instance1.num`（它们都指向堆上同一个对象，所以 `instance1.num` 和 `instance2.num` 都是 5）。这本身不是 bug，但需要理解引用类型的这个特性。

蓝色代码（需要检查的写法）：

```
A instance1 = new A();
int num = instance1.num; // num = 0 (int默认值)
num = 5;                 // 只修改了本地int变量
```

这里 `num` 是一个独立的本地 `int`（值类型）变量。`num = 5` 只是把本地变量设置为 5，与 `instance1.num` 没有任何关联。`instance1.num` 仍然是 0。

结论：如果需要修改 `instance1.num`，必须写 `instance1.num = 5`，不能通过中间变量间接修改。

11.3 题 3: *StringBuilder* 的使用

为什么“+”运算符拼接字符串不是好选择？

由于字符串的不变性（**Immutability**），每次使用 + 拼接字符串时都会在堆上创建一个全新的字符串对象，旧的字符串变成垃圾。在循环中使用尤其糟糕——1000 次拼接产生约 1000 个垃圾对象，触发频繁 GC，严重影响性能。

使用 **StringBuilder** 拼接字符串：

```
using System.Text;

// 将一个string数组中的元素依次拼接为一个字符串
string ConcatenateStrings(string[] strings) {
    StringBuilder sb = new StringBuilder();
    foreach (string s in strings) {
        sb.Append(s);
    }
    return sb.ToString();
}

// 使用示例
string[] words = { "Hello", " ", "World", "!" };
string result = ConcatenateStrings(words);
// result = "Hello World!"
```

StringBuilder 的工作原理： *StringBuilder* 内部维护一个可变的字符缓冲区。**Append()** 方法直接将新字符写入缓冲区的尾部，不会创建中间字符串对象。当调用 **ToString()** 时，一次性从缓冲区构建出最终的不会变的字符串。无论拼接多少次，只在末尾产生一个字符串对象。

性能对比：

- 10 次拼接：+ 运算符产生约 9 个垃圾，*StringBuilder* 产生 1 个——差异不大
- 10000 次拼接：+ 运算符产生约 9999 个垃圾，可能触发多次 GC；*StringBuilder* 仍然只产生 1 个——差异巨大

使用建议：当拼接次数超过 3-5 次，或拼接在循环中进行时，使用 **StringBuilder**；简单的 2-3 个字符串用 + 或字符串插值（`$"a{b}"`）即可，它们经过编译器优化后会调用 **string.Concat()**，不会产生中间垃圾。